Lecture
Developing Android Apps

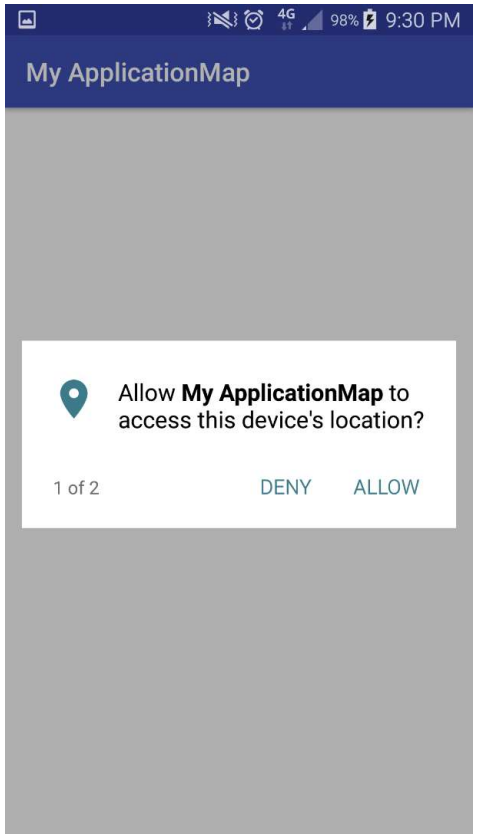
BroadCastReciever + Service

Dr. Murad Mustafa Badarna

Requesting Permissions at Run Time

Work with minSdkVersion 23

- Beginning in Android 6.0 (API level 23), users grant permissions to apps while the app is running, not when they install the app.
- This approach streamlines the app install process, since the user does not need to grant permissions when they install or update the app.
- System permissions are divided into two categories, *normal* and *dangerous:*
  - Normal permissions do not directly risk the user's privacy. If your app lists a normal permission in its manifest, the system grants the permission automatically.
  - Dangerous permissions can give the app access to the user's confidential data.
- If your app lists a normal permission in its manifest, the system grants the permission automatically.
- If you list a dangerous permission, the user has to explicitly give approval to your app.

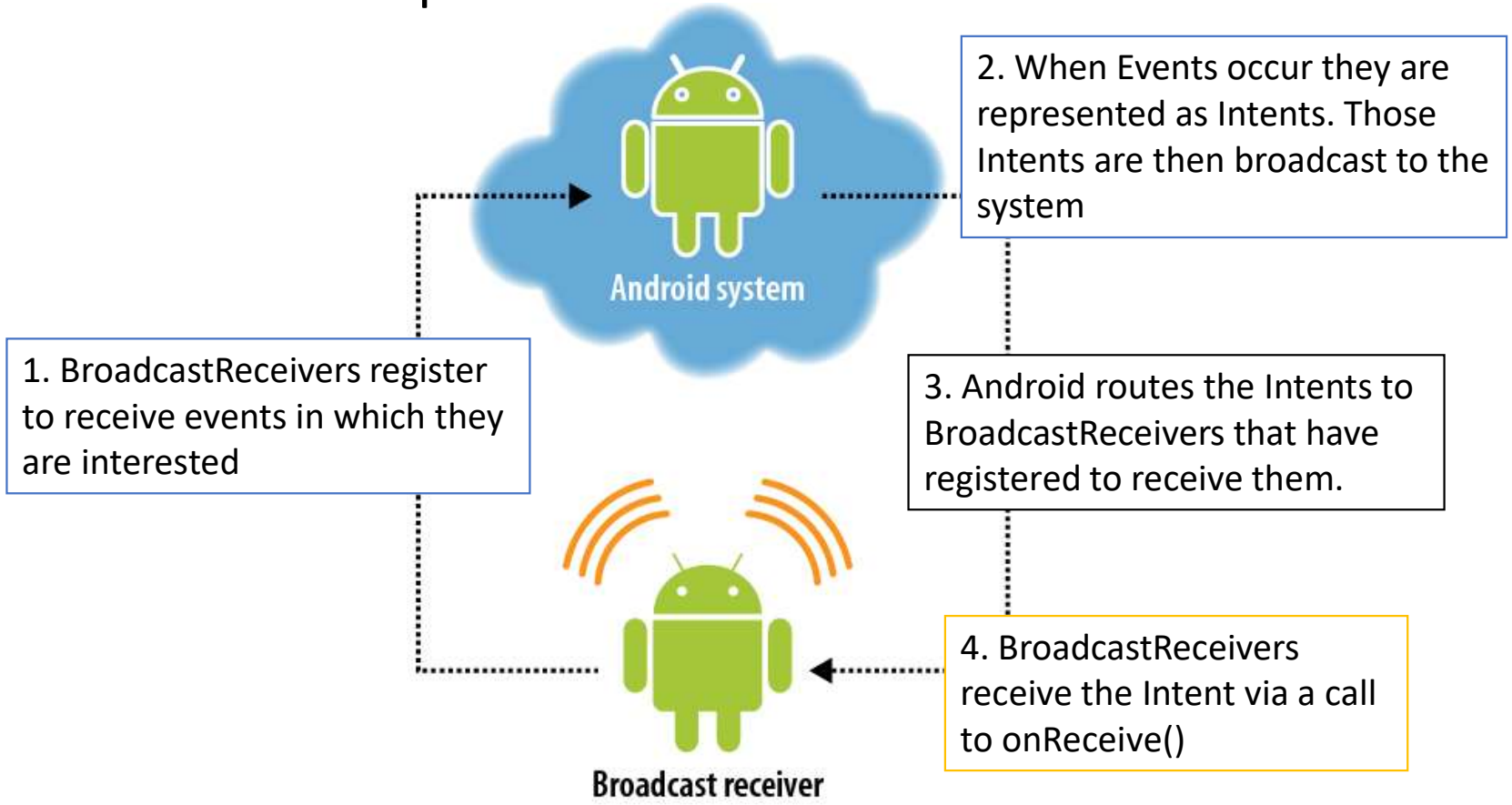


Import project called **MaterialApp2.zip** (download from our course website) into Android Studio

BroadcastReceiver

BroadcastReceiver הוא רכיב במערכת אנדרואיד שמאפשר לאפליקציות להאזין לאירועים מערכתיים או אפליקטיביים – כמו שינוי בחיבור לרשת, סיום הטעינה של המכשיר, או הודעה חדשה שהתקבלה.

- Base class for components that receive and react to events



BroadcastReceiver registration

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="package.name.test"
    android:versionCode="1"
    android:versionName="1.0" >

     <uses-sdk android:minSdkVersion="10" />

    <uses-permission android:name="android.permission.READ_PHONE_STATE" >
    </uses-permission>

    <application
        android:icon="@drawable/icon"
        android:label="@string/app_name" >
        <receiver android:name="TestPhoneReceiver" >
            <intent-filter>
                <action android:name="android.intent.action.PHONE_STATE" >
                </action>
            </intent-filter>
        </receiver>
    </application>
</manifest>
```

put <receiver> and <intent-filter> tags in AndroidManifest.xml
Register to receive events in which they are interested

See BroadcastReceiverExample App

BroadcastReceivers receive the Intent via a call to onReceive()

```
public class TestPhoneReciver  extends BroadcastReceiver {
   @Override
   public void onReceive(Context context, Intent intent) {
      Bundle extras = intent.getExtras();
      if (extras != null) {
         String state = extras.getString(TelephonyManager.EXTRA_STATE);
         Log.d("TEST", state);
         if (state.equals(TelephonyManager.EXTRA_STATE_RINGING)) {
            String phoneNumber = extras
                  .getString(TelephonyManager.EXTRA_INCOMING_NUMBER);
            Log.d("TEST", phoneNumber);
         }
      }
   }
}
```

Receive intent whenever device is booted

```
public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Intent Detected.", Toast.LENGTH_LONG).show();
    }
}
```

```
<application
  android:icon="@drawable/ic_launcher"
  android:label="@string/app_name"
  <receiver android:name="MyReceiver">
    <intent-filter>
      <action android:name="android.intent.action.BOOT_COMPLETED">
      </action>
    </intent-filter>
  </receiver>
</application>
```

Important system events

Event Constant	Description
android.intent.action.BATTERY_CHANGED	Sticky broadcast containing the charging state, level, and other information about the battery.
android.intent.action.BATTERY_LOW	Indicates low battery condition on the device.
android.intent.action.BATTERY_OKAY	Indicates the battery is now okay after being low.
android.intent.action.BOOT_COMPLETED	This is broadcast once, after the system has finished booting.
android.intent.action.CALL	Perform a call to someone specified by the data.
android.intent.action.CALL_BUTTON	The user pressed the "call" button to go to the dialer or other appropriate UI for placing a call.
android.intent.action.DATE_CHANGED	The date has changed.
android.intent.action.REBOOT	Have the device reboot.

See sendBroadCast App example

Service

Service הוא רכיב באפליקציית אנדרואיד שמיועד לביצוע פעולות ממושכות או ברקע ללא ממשק משתמש . זהו פתרון אידיאלי לפעולות שצריכות להמשיך גם כאשר המשתמש עוזב את האפליקציה הראשית.

- An application component that can perform long-running operations in the background.
- It does not provide a user interface.
- Another application component can start a service, and it will continue to run in the background even if the user switches to another application.
- Example usage:
  - Handle network transactions from the background.
  - Plays music file using a service.
  - Web browser runs a downloader service to retrieve a file.

•ניתן להפעיל אותו מרכיב אחר Activity, BroadcastReceiver וכו והוא ימשיך לפעול גם אם המשתמש יעבור לאפליקציה אחרת.
•ניתן להשתמש ב־ Serviceלצורך משימות כמו:
•ביצוע בקשות לרשת (API calls) ברקע.
•השמעת מוזיקה גם כאשר האפליקציה סגורה.
•הורדת קובץ תוך כדי שימוש באפליקציות אחרות.

Creating a Service

override the methods **onCreate(), onStartCommand()**, and **onDestroy().**

```
public class HelloService extends Service {

  @Override
  public void onCreate() {
    // Start up the thread running the service.  Note that we create a
    // separate thread because the service normally runs in the process's
    // main thread, which we don't want to block.  We also make it
    // background priority so CPU-intensive work will not disrupt our UI.
  }

  @Override
  public int onStartCommand(Intent intent, int flags, int startId) {
      Toast.makeText(this, "service starting", Toast.LENGTH_SHORT).show();

      // If we get killed, after returning from here, restart
      return START_STICKY;
  }


  @Override
  public void onDestroy() {
    Toast.makeText(this, "service done", Toast.LENGTH_SHORT).show();
  }
}
```

Creating a Service

- Creating a Service in Android involves extending the Service class and adding a service tag to the **AndroidManifest.xml**.

```
<manifest ... >
  ...
  <application ... >
      <service android:name=".HelloService" />
      ...
  </application>
</manifest>
```

note that you must write a dot (.) before the class name

- you can ensure that your service is available to only your app by including the **android:exported** attribute and setting it to "**false**". This effectively stops other apps from starting your service.

Starting a Service

- You can start a service from an activity or other application component by passing an Intent to **startService().**
- The Android system calls the service's **onStartCommand()** method and passes it the Intent. (You should never call **onStartCommand()** directly.)
- The **startService()** method returns immediately and the Android system calls the service's **onStartCommand()** method.
- If the service is not already running, the system first calls **onCreate()**, then calls **onStartCommand().**

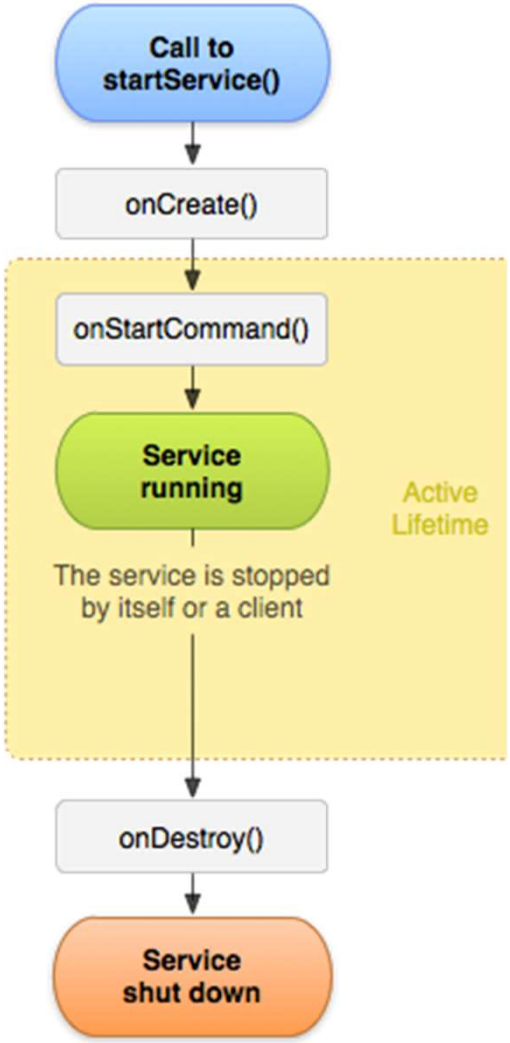
```
Intent intent = new Intent(this, HelloService.class);
startService(intent);
```

Stopping a service

- A service must manage its own lifecycle.
- The system does not stop or destroy the service unless it must recover system memory and the service continues to run after `onStartCommand()` returns.
- So, the service must stop itself by calling `stopSelf()` or another component can stop it by calling `stopService()`.
- Once requested to stop, the system destroys the service as soon as possible.

Service lifecycle

- Like an activity, a service has lifecycle callback methods that you can implement to monitor changes in the service's state and perform work at the appropriate times.
- The entire lifetime of a service happens between the time **onCreate()** is called and the time **onDestroy()** returns.
- Like an activity, a service does its initial setup in **onCreate()** and releases all remaining resources in **onDestroy().** For example, a music playback service could create the thread where the music will be played in onCreate(), then stop the thread in **onDestroy().**
- The active lifetime of a service begins with a call to **onStartCommand()**. This method is handed the Intent that was passed to **startService()**.
- If the service is started, the active lifetime ends the same time that the entire lifetime ends (the service is still active even after **onStartCommand()** returns).



המתודה onStartCommand() חייבת להחזיר ערך שלם קבוע
שמנחה את מערכת אנדרואיד **מה לעשות אם ה־ Service נעצר** על ידי המערכת

Service states

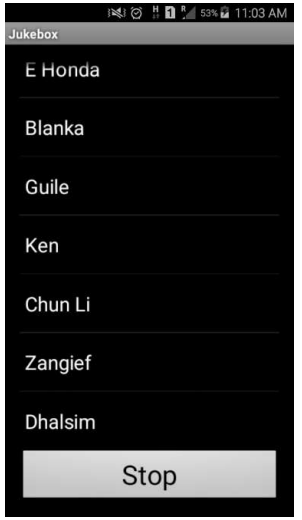
- Notice that the `onStartCommand()` method must return an integer. The integer is a value that describes how the system should continue the service in the event that the system kills it
  - **START_NOT_STICKY**
    - If the system kills the service after **onStartCommand**() returns, **do not recreate the service**, unless there are pending intents to deliver. This is the safest option to avoid running your service when not necessary and when your application can simply restart any unfinished jobs.
  - **START_STICKY**
    - If the system kills the service after **onStartCommand**() returns, **recreate the service** and call **onStartCommand**(), but do not redeliver the last intent. Instead, the system calls **onStartCommand**() with a null intent, unless there were pending intents to start the service, in which case, those intents are delivered. **This is suitable for media players** (or similar services) that are not executing commands, but running indefinitely and waiting for a job.
  - **START_REDELIVER_INTENT**
    - If the system kills the service after **onStartCommand**() **returns, recreate the service and call onStartCommand() with the last intent that was delivered to the service**. Any pending intents are delivered in turn. This is suitable for services that are actively performing a job that should be **immediately resumed, such as downloading a file**.

השתמשו ב־ START_NOT_STICKY כאשר לא צריך להפעיל מחדש את השירות אוטומטית.
בחר ב־ START_STICKY אם השירות "ממתין" למשימות.
בחר ב־ START_REDELIVER_INTENT אם השירות **חייב להשלים משימה קריטית** שהתחיל.

Example (Media Player using Service)

```
        <service
            android:name=".JukeboxService"
            android:enabled="true"
            android:exported="false" >
        </service>
    </application>

</manifest>
```

```
public class JukeboxService extends Service {
    // we declare constants for "actions" that are packed into each intent
    public static final String ACTION_PLAY = "play";
    public static final String ACTION_STOP = "stop";

    // the actual media player to play the music
    private MediaPlayer player;

    /*
     * This method is called each time a request comes in from the app
     * via an intent.  It processes the request by playing or stopping
     * the song as appropriate.
     */
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {

        if(intent!=null){
                String action = intent.getAction();
                try {
                    if (action.equals(ACTION_PLAY)) {
                        // convert the song title into a resource ID
                        // e.g. "ehonda" -> R.raw.ehonda
                        String title = intent.getStringExtra("title");
                        int id = getResources().getIdentifier(title, "raw", getPackageName());
```

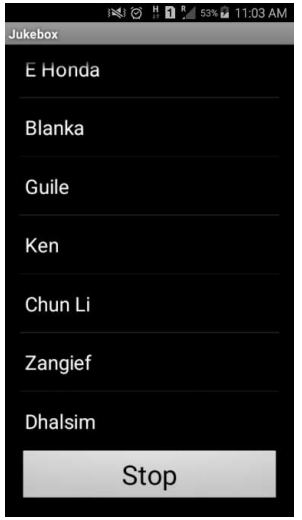


Lets look at the code of the demo app.

Import project called **jukebox1.zip** (download from our course website) into Android Studio

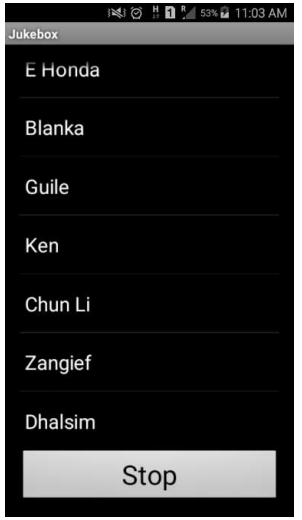
Example (Media Player using Service)

```
                    // stop any previous playing song
                    if (player != null && player.isPlaying()) {
                        player.stop();
                        player.release();
                        player = null;
                    }

                    // start the new song
                    player = MediaPlayer.create(this, id);
                    player.setLooping(false);
                    player.start();
                } else if (action.equals(ACTION_STOP)) {
                    // stop any currently playing song
                    if (player != null && player.isPlaying()) {
                        player.stop();
                        player.release();
                        player = null;
                    }
                }
            } catch (IllegalStateException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }

        return START_STICKY;   // START_STICKY means service will keep running
    }
```



Example (Media Player using Service)

```
public class JukeboxActivity extends Activity
        implements AdapterView.OnItemClickListener {

    /*
     * This method runs when the activity is started up.
     * Sets up initial state of the GUI.
     */
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_jukebox);

        // listen for clicks on list items
        ListView list = (ListView) findViewById(R.id.song_list);
        list.setOnItemClickListener(this);
    }

    /*
     * This method is called when items in the ListView are clicked.
     * It initiates a request to the JukeboxService to play the given song.
     */
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int index, long id) {
        String[] songTitles = getResources().getStringArray(R.array.song_titles);
        String title = songTitles[index].toLowerCase().replace(" ", "");
```



Example (Media Player using Service)

```
     * This method is called when items in the ListView are clicked.
     * It initiates a request to the JukeboxService to play the given song.
     */
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int index, long id) {
        String[] songTitles = getResources().getStringArray(R.array.song_titles);
        String title = songTitles[index].toLowerCase().replace(" ", "");

        Intent intent = new Intent(this, JukeboxService.class);
        intent.putExtra("title", title);
        intent.setAction(JukeboxService.ACTION_PLAY);
        startService(intent);
    }



    /*
     * This method is called when the Stop button is clicked.
     * It initiates a request to the JukeboxService to stop the current playback.
     */
    public void onClickStop(View view) {
        Intent intent = new Intent(this, JukeboxService.class);
        intent.setAction(JukeboxService.ACTION_STOP);
        startService(intent);
    }
}
```